

Lecture Notes on **Machine Learning Theory**

Owen Lin
owen2006lin@gmail.com

These notes are for a graduate level course on machine learning theory, with an emphasis on statistical learning theory and online learning.

- Shalev-Schwartz and Ben-David: *Understanding Machine Learning: From Theory to Algorithms*
- Blum, Hopcroft, Kannan: *Foundations of Data Science*

Contents

1	Basics?	3
1.1	Premise of Learning	3
1.2	Empirical Risk Minimization	3
1.3	PAC Learning	5
1.4	Sample Complexity	6

1 Basics?

1.1 Premise of Learning

To begin, we'll present some illustrative examples. Suppose we desire to train a model to classify spam e-mails.

- A naive approach is to simply memorize all previous e-mails that have been classified as spam by a human user. Then, when a new email arrives, we search for this e-mail in the dataset of all past e-mails and classify accordingly. A problem with this “**learning by memorization**” is that this doesn't generalize well to data outside of the original dataset.
- A better approach is to look at the previous set of all emails, and then extract a set of words that correlates strongly with classification. Then we apply this rule for all future e-mails. “**Inductive reasoning**” allows us to generalize to data outside of the original dataset.
- Inductive reasoning may lead us to false conclusions. An example is the **Pigeon Superstition**. The experiment placed food at random intervals initially. Pigeons associated whatever action they did when food arrived and engaged in that activity more. Over time, they began to associate the action with the food delivery even though the true pattern was random.

The goal of course, is to develop performant learners. Human learners can filter out this pigeon superstition from common sense, and we'd desire principles that allow a learner to reject useless conclusions. One powerful tool is the incorporation of **prior knowledge** to bias the learning mechanism. The idea is to develop domain specific knowledge, and then translate it into a way to bias the learning to agree.

1.2 Empirical Risk Minimization

First, we'll lay down some formal notation for learning. In the classical setting:

- The learner has access to several datasets for inputs. The **domain set** X contains the set of objects to label. Each point may be represented using a vector of features. The **label set** y contains all possible labels, for now we'll restrict it to a two element set: $y = \{-1, +1\}$. Finally, the **training data** S consists of a set of (x_m, y_m) pairs, the training examples the learner has access to. Namely, $S = ((x_1, y_1) \dots (x_m, y_m))$.
- The learner should output a prediction rule $h: X \rightarrow y$ that can be used to predict the label of domain points. The function is also known as a **hypothesis** or a **predictor**. We also write this hypothesis of some output of our learning algorithm A upon receiving training sample S .
- To generate the data, we sample some points in X with probability distribution D . Then, we use the true labeling function $f: X \rightarrow y$, and create labels $y_i = f(x_i)$.
- The error L of a classifier is the probability it does not predict the correct label of a random data point over D . Namely,

$$L_{D,f}(h) \equiv \mathbb{P}_{x \sim D}[h(x) \neq f(x)] \equiv D(\{x : h(x) \neq f(x)\})$$

L is also called the **generalization error** or the **risk** or the **true error**. How we use the distribution D is, if we have some subset $A \subset X$ then $D(A)$ gives us how likely it is to observe a point $x \in A$.

A good learner should try to minimize the error with respect to some unknown D and truth f . Because the learner doesn't know D and f , we can use a heuristic for the loss called the **training error**:

$$L_S(h) \equiv \frac{|\{i \in [m] : h(x_i) \neq y_i\}|}{m}$$

This paradigm, in finding a predictor h to minimize the training error is **Empirical Risk Minimization**.

- Overfitting is obviously a big problem. A hypothesis that works very well on the training set can generalize very poorly over all data. Additionally, there may exist multiple possible hypothesis that perform equally on the training set, and choosing one is unclear.
- A solution is to restrict ERM over a finite set of hypotheses, also called the **hypothesis class** $h \in H$. This is a form of inductive bias, but is not always sufficient to guarantee we don't overfit.
- The **Realizability Assumption** states there exists $h^* \in H$ such that $L_{(D,f)}(h^*) = 0$. More formally, we say that a distribution D is realizable with respect to hypothesis class H if there exists a perfect hypothesis with no true error. In other words, the solution for our distribution belongs to our hypothesis class.
- The **i.i.d assumption** assumes the examples in the training set are independently and identically distributed. We denote $S \sim D^m$ where D^m is the probability over m -tuples by applying D to pick each element of the tuple independently.

The process for empirical risk minimization is as follows: we've narrowed down our set of hypotheses to a hypothesis class H . We want to minimize the empirical risk over a training set S , and can choose to assume the realizability assumption. If we do, then ERM selects any hypothesis h that achieves 0 training error. Note that if we don't assume the realizability assumption, it's not guaranteed that this will be the case, leading us into **Agnostic Learning**.

Some important metrics to keep in mind are **confidence** and **accuracy**. We can denote the probability of sampling a training set is not representative of the underlying D using the confidence $(1 - \delta)$. The accuracy parameter can be denoted as ϵ , where we note a training failure when $L_{(D,f)}(h_S) \geq \epsilon$

- We'd like to upper bound the probability that some sample will lead to learning failure. Namely, we want to find the bound for

$$D^m(\{S|_x : L_{(D,f)}(h_s) > \epsilon\})$$

- By the realizability assumption, ERM gives us a hypothesis h_s such that $L_S(h_s) = 0$, so the only time learning fails is when our sample is in the set of misleading samples, where there exists hypothesis with 0 training error, but nonzero generalization error. The set of all such samples is also called M , so we wish to solve

$$D^m(\{S|_x : L_{(D,f)}(h_s) > \epsilon\}) \leq D^m(M)$$

- Since each element is sampled i.i.d, the probability a sampled point is one where a bad hypothesis looks correct is $1 - L_{(D,f)}(h)$, which is less than $1 - \epsilon$. If we note that $1 - \epsilon \leq e^{-\epsilon}$ and note that each element is independent, then multiplying the probability over all points in the training set, the probability that a bad hypothesis leads to failure over a sample is

$$D^m(\{S|_x : L_s(h) = 0\}) \leq (1 - \epsilon)^m \leq e^{-\epsilon m}$$

Applying the union bound inequality, we have

$$D^m(\{S|_x : L_{(D,f)}(h_s) > \epsilon\}) \leq |H|e^{\epsilon m}$$

- For a finite hypothesis class, if we have an integer m such that $m \geq \frac{\log(|H|/\delta)}{\epsilon}$, then for any labeling function f and distribution D where the realizability assumption holds, we have probability of at least $1 - \delta$ that we have a bad hypothesis:

$$Pr_{S \sim D^m}[L_{(D,f)}(h_S) \leq \epsilon] \geq 1 - \delta$$

In other words, for sufficiently large m , the ERM rule over a finite hypothesis class will be probably (with confidence $1 - \delta$) approximately (up to error ϵ) correct.

1.3 PAC Learning

In this section, we concern ourselves with hypotheses that are **P**robably **A**pproximately **C**orrect.

We say a hypothesis class H is **PAC learnable** if there exists a function $m_H : (0, 1)^2 \rightarrow \mathbb{N}$ and a learning algorithm obeying: $\forall \epsilon, \delta \in (0, 1)$, for every distribution D over X and for every labeling function $f : X \rightarrow \{0, 1\}$, if the realizability assumption holds w.r.t. H, D, f , then when A is given $m \geq m_H(\epsilon, \delta)$ i.i.d examples drawn from D and labeled with f , it returns a hypothesis h satisfying

$$err_{D,f}(h) \leq \epsilon$$

with probability at least $1 - \delta$ over random training sample. we call m_H the **sample complexity** of learning H , basically it specifies how many training examples are sufficient to guarantee a PAC solution. Some things to note:

- You may wonder why an algorithm can't just find the perfect solution each time if it belongs to the hypothesis class anyways. Because the training set is randomly generated, it might not be representative of the distribution D . For example, a case could be that the same point is simply resampled over and over again.
- Even if the sample is representative of D , the size is finite and less than the whole domain. Thus, there may be some details of D that simply can't be captured by the sample. We introduce accuracy parameter ϵ to allow for these minor mistakes.
- If we assume that S is perfect and of good size, we're still not guaranteed to find the best hypothesis, since our learning algorithm might not be deterministic. Thus, we introduce the confidence parameter δ to describe how likely we're able to reach the target accuracy.

If we note the theorem from above, every finite hypothesis class is actually PAC learnable, with sample complexity

$$m_H(\epsilon, \delta) \leq \left\lceil \log \left(\frac{(|H|/\delta)}{\epsilon} \right) \right\rceil$$

Example Consider the class of all one-dimensional thresholds. Let the domain be $\mathbb{R}$, and restrict our hypothesis class H to be all thresholds of form

$$h_a(x) = \begin{cases} 1, & \text{if } x \leq a \\ 0, & \text{if } x \geq a \end{cases}$$

Assume the realizable setting, and assume that our learned threshold $\hat{a}$ is less than the true threshold a^* . Choose a point a_L to the left of a^* such that the probability mass of the interval $(a_L, a^*]$ is exactly ϵ . If our algorithm picks a threshold $\hat{a}$ to the left of a_L , then the generalization error will be greater than ϵ . Now for this to happen, all m training samples must miss this interval, because if one fell in the correct interval, ERM would automatically select $\hat{a}$ to fall under the $< \epsilon$ range. The probability this happens is $(1 - \epsilon)^m$. Applying this logic to the other side, we have $\mathbb{P}(L_D(h_{\hat{a}})) \leq 2(1 - \epsilon)^m$. If we let this failure probability be at most δ , then rearranging gives us

$$m \geq \frac{1}{\epsilon} \ln \left(\frac{2}{\delta} \right)$$

Example: For a finite hypothesis class, we've shown that the upper bound of the probability that some sample will fail is $\delta \geq |H|e^{-m\epsilon}$. Rearranging, the lower bound for the sample size needed to achieve ϵ error with probability $1 - \delta$ is

$$m \geq \frac{1}{\epsilon} \left(\ln(|H|) + \ln \left(\frac{1}{\delta} \right) \right)$$

1.4 Sample Complexity

The **sample complexity** m is the number of training data examples an algorithm needs to successfully learn a target function. Note that for an infinite hypothesis class size, these bounds that we found for the sample complexity are vacuous. In this case, we instead focus on the number of ways to label samples rather than on the size of a hypothesis class.

We define the **Growth Function** as follows: For an unlabeled sample $T = \{x_1, x_2, \dots, x_n\}$, let $H[T]$ denote the set of all ways (sequences) to label our sample using hypotheses from H . Namely, $H[T] = \{h(x_1), \dots, h(x_n) : h \in H\}$. Then we define the growth function $\Pi_H(n)$ as

$$\Pi_H(n) = \max_{T \in X^n} |H[T]|$$

Colloquially, it's the number of ways to label an n size sample using hypotheses from class H . For

binary classification, the max is trivially 2^n .

Example: for the class of one dimensional thresholds, the growth function is $n + 1$. This is because to choose a labeling, we must choose a point a^* such that any points left of this is 0, and any point to the right is 1. For n points, there are $n + 1$ spaces we can choose for a^* .

We can use the growth function as a replacement for the size of our hypothesis class. Any hypothesis class is PAC learnable if the sample complexity satisfies

$$m \geq \frac{2}{\epsilon} \left(\log_2 (2\Pi_H(2m)) + \log_2 \left(\frac{1}{\delta} \right) \right)$$

The proof is quite long, and linked [here](#). Some clever tricks to take away:

- We can upper bound the probability that Bernoulli random variables using **Chernoff Bound**. If we let $X_1, \dots, X_m$ be a sequence of Bernoulli random variables with expectation μ , the probability they are far from the mean is

$$\Pr \left[\frac{1}{m} \sum_{k=1}^m X_k \leq \frac{\mu}{2} \right] \leq \exp \left(-\frac{m\mu}{8} \right)$$

Meanwhile, for any $\epsilon > 0$, **Hoeffding's bound** gives us

$$\Pr \left[\left| \frac{1}{m} \sum_{k=1}^m X_k - \mu \right| \geq \epsilon \right] \leq 2\exp(-2m\epsilon^2)$$

- Using a **ghost sample**. Still don't understand this one...